

HANDOVER DOCUMENT TASK 6 - SOKOBAN MULTIPLAYER

1. Two-Player Map (map_2p.txt)

The map is designed with an 11x9 dimension, completely symmetrical to ensure absolute fairness.

```
source > maps > map_2p.txt
 1  %%%%%%%%%%
 2  %1          2%
 3  %D  %  %  D%
 4  %   B B   %
 5  %D%   B   %D%
 6  %   B B   %
 7  %D  %  %  D%
 8  %           %
 9  %%%%%%%%%%
```

Symbol Convention (Legend):

- %: Wall - Impassable.
- (space): Floor - Valid movement area.
- B: Box - Pushable object (5 boxes in total).
- D: Goal - Target location to push boxes into (6 goals in total).
- 1: Starting position of Agent 1 (Left).
- 2: Starting position of Agent 2 (Right).

2. Core Gameplay Rules (Box-Stealing Mechanism)

- Real-time: The game operates in real-time. At each step of the game, both Agents must calculate and execute their actions (Move or Push) simultaneously.
- Scoring & Box-Stealing Mechanism: When an Agent pushes a box B onto a goal D, that Agent temporarily scores a point. However, the box is not locked. The opposing Agent has the full right to push that box off the goal and maneuver it to their own goal (stealing points).
- Collision: The two Agents are not allowed to pass through each other. (Note: You need to implement additional logic to handle scenarios where both rush into the same empty cell or

attempt to push the same box at the same time).

- Termination Condition: The game does not end based on the number of remaining boxes. It will automatically stop after n steps (turn limit) entered by the user at the beginning.
- Winning Condition: Upon reaching the n-step limit, the Agent with the most boxes resting on the goals is the winner.

3. Problem Modeling (Algorithm Coding Support)

The assignment requires you to implement each Agent in an independent code file. You can use fundamental algorithms you have learned (such as BFS, UCS, DFS, DLS, IDS, GBFS, or A*) for each Bot.

- General Direction: This is a Multi-Agent Pathfinding (MAPF) problem. Each Bot will operate as an independent entity, continuously scanning the current map state to make decisions.
- Objective Function for each Bot:
 - Find a path (using an algorithm like A*) to the nearest box B.
 - After reaching a box, find the shortest path to push it to the nearest goal D (even if that goal currently holds the opponent's box).
- Search Space Constraints:
 - Do not pass through walls (%).
 - Treat the opponent's current position as a temporary obstacle.
 - Only 1 box can be pushed at a time, provided the cell directly behind the box is empty or a goal.
- Time Limit (Timeout): The assignment specifies a maximum of 1000ms for each Agent to make a decision (choose 1 action) per step. Remember to limit your algorithm's search depth or use `time.time()` to interrupt the function if the pathfinding process takes too long.

4. Map Design Tactical Notes (For Reporting)

- Fairness: The distances from the starting positions of the 2 Agents to the boxes and goals are perfectly equal. There is no "home-field" advantage.
- "Box-Stealing" Incentive: An open map design with few dead ends and a higher number of goals (6) compared to boxes (5) encourages continuous interaction. The fact that boxes are not locked at the goals forces both AIs to keep moving and fighting for already-placed boxes, rather than remaining idle and waiting for the n steps to run out.